Sprint 26 test pack

Sprint 26 test pack: protecting the branch that publishes the live demo

What this is. S26-2 (F-77): after these steps, only an approved production deploy, or you, can move `live`, the branch Vercel publishes as the live demo. You run one script on your PC; it changes the repository's settings in GitHub. No app, no test accounts and no data are involved, and nothing here is secret: the private key goes straight from the script into GitHub and is never shown.

About 15 minutes.

Already checked by Claude: - The script's tests (rulesets as agreed, the key's private half only ever passed through standard input, refusal if another deploy key can write, a failed store undoing itself). - The deploy workflow's tests: its token can only read, it pushes `live` with the key to GitHub's pinned host key, and it deploys nothing unless the key is set, the rules on `live` are in force, and `production` deploys from `main` only. The push step itself was run with a stand-in for `git`. - What cannot be checked before your set-up: the push with the real key (at the first production deploy after the merge) and GitHub refusing everyone else (cases 1 and 2 of the refinement, which Claude runs after you).

Before you start

In PowerShell, in `C:\Code\rfi-responder`. Claude has put your copy on the branch `sprint-26`, which has the script.

1. `git branch --show-current` says `sprint-26`.
2. `gh auth status` says you are signed in to github.com as `rorymcclelland`.
3. `ssh-keygen -?` prints its usage (Windows includes it; if not: Settings, Apps, Optional features, add **OpenSSH Client**).
4. **Do not approve** the production run that is waiting for the Sprint 25 close (Actions, Deploy, run 42386689435). Once `live` is protected that older workflow could not move it, and the run would fail after deploying the functions. Choose **Reject** for it, or leave it.

Case 1. The production environment deploys from `main` only

What it checks: S26-2 case 5, the setting the deploy now requires.

Flow: 1. In GitHub, open the repository, then **Settings**, **Environments**, **production**. 2. Under **Deployment branches and tags**, choose **Selected branches and tags** if it is not already. 3. The list holds exactly one rule: **main**, of type **Branch**. Add it if missing, and remove any other.

Expected outcome: Selected branches and tags, with the one rule `main` (Branch). **Required reviewers** still lists you.

Result: ☐ Pass ☐ Fail. Notes: _____

Case 2. A dry run

What it checks: that the script can read the repository, and what it will do.

Flow: `npx tsx scripts/setup-live-key.ts`

Expected outcome:

```
Dry run for the protection of live and main in rorymcclelland/RFI-
responder.
  ruleset "live: only the approved deploy": would be made
  ruleset "main: no force push or deletion": would be made
  deploy key: a new one would be made, its private half stored as
LIVE_DEPLOY_KEY in the production environment
Nothing was changed. Run again with --apply to do it.
```

If it says **Refusing: … other deploy key(s) can write**, stop and send Claude that line.

Result: ☐ Pass ☐ Fail. Notes: _____

Case 3. Apply it

What it checks: S26-2's set-up: both rulesets, and the key.

Flow: `npx tsx scripts/setup-live-key.ts --apply`

Expected outcome:

```
Setting up the protection of live and main in rorymcclelland/RFI-
responder.
  ruleset "live: only the approved deploy": made
  ruleset "main: no force push or deletion": made
  deploy key: made, and its private half stored as LIVE_DEPLOY_KEY
in the production environment
```

No line says **not set** or **refused**, and nothing that looks like a key is printed.

Result: ☐ Pass ☐ Fail. Notes: _____

Case 4. What GitHub now shows

What it checks: S26-2 cases 6 and 7, by eye (Claude also reads the rules through GitHub's API).

Flow and expected outcome: 1. **Settings**, **Rules**, **Rulesets**: two rulesets, both **Active**: **live: only the approved deploy** (targets `live`; restricts updates, deletions and force pushes; bypass: deploy keys and the Repository admin role) and **main: no force push or deletion** (targets `main`; no bypass). 2. **Settings**, **Deploy keys**: one key, **RFI Respond: the approved deploy moves live** with today's date, with **Read/write**. 3. **Settings**, **Environments**, **production**: the secrets include **LIVE_DEPLOY_KEY**.

Result: ☐ Pass ☐ Fail. Notes: _____

Afterwards

Send Claude "done", or the case numbers that failed with a screenshot. Claude then reads the rules on `live` through GitHub's API, tries to move `live` from a throwaway `sprint-*` branch's workflow and with its own credentials (both must be refused), and records the results. The push with the key is shown at the first production deploy after the Sprint 26 merge, which you approve as usual.

If the key is ever lost or may have leaked, run the script again with `--apply`: it makes a new key and removes the old one.